
ALGORITMOS E ESTRUTURAS DE DADOS EM JAVASCRIPT

Do HTML estático às aplicações modernas: Uma jornada técnica pela evolução da web.

</> Fundamentos de Programação Web



A EVOLUÇÃO DA WEB: DO DOCUMENTO À APLICAÇÃO

A Origem e o Dinamismo

1990 Tim Berners-Lee cria o HTML para compartilhar documentos acadêmicos estáticos.

1995 Brendan Eich desenvolve o JavaScript em 10 dias para dar vida às páginas web.

2009 O Node.js expande o JS para o servidor, tornando-o onipresente no desenvolvimento.

A Tríade Clássica do Front-end



HTML

Estrutura e conteúdo da página.



CSS

Aparência, layout e estilo visual.



JavaScript

Comportamento, lógica e interatividade.

ALGORITMOS: O "COMO FAZER" DA PROGRAMAÇÃO

Um algoritmo é uma sequência finita de instruções para resolver um problema. Na computação, eles descrevem o passo a passo da ação da máquina, construídos sobre três pilares fundamentais:

↓ Sequência

Instruções executadas uma após a outra em ordem linear.

```
const total = preco - desc;  
console.log(total);
```

🔗 Condição

O programa toma decisões com base em critérios lógicos.

```
if (idade >= 18) {  
  permitir();  
}
```

↻ Repetição

Ações repetidas enquanto uma condição for verdadeira.

```
for (let i=0; i<5; i++) {  
  console.log(i);  
}
```

ESTRUTURAS DE DADOS: ORGANIZANDO A INFORMAÇÃO

O "Como Guardar"

Enquanto algoritmos focam na lógica da ação, as estruturas de dados definem como a informação é organizada e armazenada na memória.

-  **Armazenamento:** Define o layout físico e lógico dos dados na memória do computador.
-  **Acesso:** Determina a facilidade e a velocidade para localizar uma informação específica.
-  **Eficiência:** Impacta diretamente na performance de operações como adição e remoção.

Impacto no Desenvolvimento

A escolha da estrutura correta é o que diferencia um código escalável de um sistema lento e difícil de manter.

"Estruturas de dados organizadas evitam o caos de gerenciar múltiplas variáveis isoladas e garantem a clareza do código."

FERRAMENTAS DE ITERAÇÃO E TRANSFORMAÇÃO

forEach: Iteração Pura

Utilizado para percorrer todos os elementos de uma lista. É a escolha ideal para **efeitos colaterais**, como renderizar elementos no DOM ou registrar logs.

```
const frutas = ["maçã", "banana"];

frutas.forEach(fruta => {
  console.log(fruta);
});
```

map: Transformação de Dados

Cria um **novo array** aplicando uma função a cada elemento. Fundamental para adaptar dados de APIs antes de exibi-los na interface.

```
const precos = [100, 200];

const descontos = precos.map(p => {
  return p * 0.9;
});
// [90, 180]
```

FILTRAGEM E AGREGAÇÃO DE DADOS

SELEÇÃO

.filter()

Retorna elementos que passam em um teste. Ideal para buscas e filtros de produtos.

```
const baratos = produtos.filter(p => p.preco < 100);
```

LOCALIZAÇÃO

.find()

Retorna o primeiro item que satisfaz a condição. Busca por ID único.

```
const user = users.find(u => u.id === 42);
```

ACUMULAÇÃO

.reduce()

Reduz o array a um único valor. Usado para somar totais de carrinhos.

```
const total = itens.reduce((acc, val) => acc + val, 0);
```

POSICIONAMENTO

.findIndex()

Retorna o índice do primeiro item encontrado. Útil para remoções precisas.

```
const index = lista.findIndex(i => i.slug === 'js');
```

ALGORITMOS DE PERFORMANCE E CONTROLE

↓ Ordenação (Sort)

Reorganiza elementos de um array com base em uma função de comparação. Essencial para tabelas e rankings.

EXEMPLO

```
lista.sort((a, b) =>  
  a.preco - b.preco  
);
```

🔍 Busca Binária

Algoritmo de divisão e conquista para encontrar itens em listas ordenadas com eficiência logarítmica $O(\log n)$.

PERFORMANCE

Divide o array ao meio a cada passo, reduzindo drasticamente o tempo de busca.

🕒 Debounce

Controla a frequência de execução de uma função. Vital para campos de busca e redimensionamento de janelas.

APLICAÇÃO

Atrasa a chamada da API até que o usuário pare de digitar por 300ms.

ESTRUTURAS FUNDAMENTAIS: ARRAY E OBJETO

Array

Coleção ordenada de elementos acessados por índice numérico.

USO: LISTAS DE PRODUTOS, ITENS DE CARRINHO.

```
const nomes = ["Ana", "Bruno"];
```

Objeto (Object)

Coleção de pares chave-valor para representar entidades.

USO: PERFIL DE USUÁRIO, CONFIGURAÇÕES.

```
const user = { id: 1, nome: "Mari" };
```

Map

Dicionário flexível que aceita qualquer tipo de chave.

USO: CACHE DE REQUISIÇÕES, ASSOCIAÇÕES COMPLEXAS.

```
const cache = new Map();
```

Set

Coleção de valores únicos que rejeita duplicatas automaticamente.

USO: LISTAS DE TAGS, IDS ÚNICOS VISITADOS.

```
const tags = new Set(["js", "web"]);
```

ESTRUTURAS DE FLUXO: PILHAS E FILAS

Stack (Pilha)

LIFO: Last In, First Out

O último elemento a entrar é o primeiro a sair. Funciona como uma pilha de pratos.

- ✓ Histórico de navegação (botão voltar)
- ✓ Funções de Desfazer/Refazer (Undo/Redo)
- ✓ Pilha de chamadas (Call Stack) do JS

Queue (Fila)

FIFO: First In, First Out

O primeiro elemento a entrar é o primeiro a ser processado. Como uma fila de banco.

- ✓ Fila de requisições HTTP
- ✓ Processamento de eventos (Event Loop)
- ✓ Sequenciamento de animações

ESTRUTURAS AVANÇADAS E O DOM

Linked List

Cadeia de nós onde cada elemento aponta para o próximo. Ideal para inserções e remoções frequentes sem reindexação.

USO PRÁTICO

Sistemas de "Desfazer" (Undo) complexos e gerenciamento de buffers.

Hash Table

Associa chaves a valores com acesso em tempo constante. Implementada em JS via Objetos e Maps.

EXEMPLO

```
contagem[palavra] = (contagem[palavra] || 0) + 1;
```

Tree (DOM)

Estrutura hierárquica fundamental. O Document Object Model é uma árvore de nós manipulável via JS.

CONCEITO

Navegar no DOM é percorrer uma árvore para modificar a interface do usuário.

COMPARATIVO DE EFICIÊNCIA (BIG O)

ESTRUTURA	ACESSO	INSERÇÃO	USO PRINCIPAL NA WEB
Array	O(1)	O(1)*	Listas e coleções ordenadas
Object / Map	O(1)	O(1)	Entidades e dicionários
Set	O(1)	O(1)	Gestão de valores únicos
Stack / Queue	O(1)	O(1)	Fluxos e históricos
Linked List	O(n)	O(1)	Estruturas dinâmicas
Tree (DOM)	O(log n)	O(log n)	Interfaces hierárquicas

● Excelente ● Bom ● Lento

* Inserção no final do array

A BASE DO DESENVOLVIMENTO MODERNO

A web evoluiu de documentos estáticos para ecossistemas complexos, mas os fundamentos permanecem inalterados. Dominar algoritmos e estruturas de dados não é apenas sobre teoria; é sobre escrever código limpo, eficiente e escalável.

 **Escolha a ferramenta certa para cada problema.**